

MAE 242: Robot Motion Planning

Programming Assignment #2

Assigned Apr 20, due May 8

Homework Instructions:

This is the second programming assignment. Please upload files on gradescope with your solutions. The Collaboration Policy for this course is detailed on the syllabus and you have to provide the names of at most 2 other students you have collaborated with to do this homework. *Please start early on this assignment, it is time consuming.*

Homework exercise:

1. (Programming problems) In this exercise, you will implement and demonstrate the performance of stochastic search algorithms in different environments. To do this, please download the folder “mdps” from canvas. In it you will find several files:

- The file `gridworld.py` defines a class `GridWorld` that supports various methods for creating, manipulating, and visualizing 2D grid environments.
The grid world is initialized from a list of obstacles, which determine the spaces where the agent cannot move. The four supported actions are “up” = $(0, 1)$, “down” = $(0, -1)$, “left” = $(-1, 0)$, or “right” = $(1, 0)$. If an action would result in collision with an obstacle, the agent remains in its current state.
When taking an action, there is a probability that another action is taken instead. This probability depends on a parameter $0 \leq \eta \leq 1$. The correct action is taken with probability $1 - \eta$, with a probability of $\frac{\eta}{2}$ of going either $\pm 90^\circ$ from the intended action. Figure ?? illustrates the potential outcomes of each action.
The function `GridWorld.nextStateRandom(x, u,  $\eta$ )` implements the Markov process described above.
- The file `grids.py` contains two example grid worlds, one small (`SmallGrid`) and one medium size (`MediumGrid`). Their format is similar to the obstacle environments in `mazemods` of the previous assignment.

You have to edit and submit the file `main.py` as your coding solution to this problem. Note that the solutions require you to deal with costs and not rewards. In other words, the values here have the interpretation of “cost to go” and the Bellman optimality equation requires a minimization, not a maximization operation.

Please submit the other results answering the following questions as a pdf file.

- (a) (10 points) Complete the function `policyIteration`. The iterations should use synchronous sweeps, and a sufficiently large number of iterations in the policy evaluation part to guarantee that v_π is practically achieved. Then use `policyIteration` to find an optimal policy and value function for `SmallGrid`. For how long you need to apply the policy evaluation and policy improvement steps until convergence with $\gamma = 0.9$ and $\eta = 0.2$? *Note:* Count the number of times you cycle over policy evaluation and policy improvement.

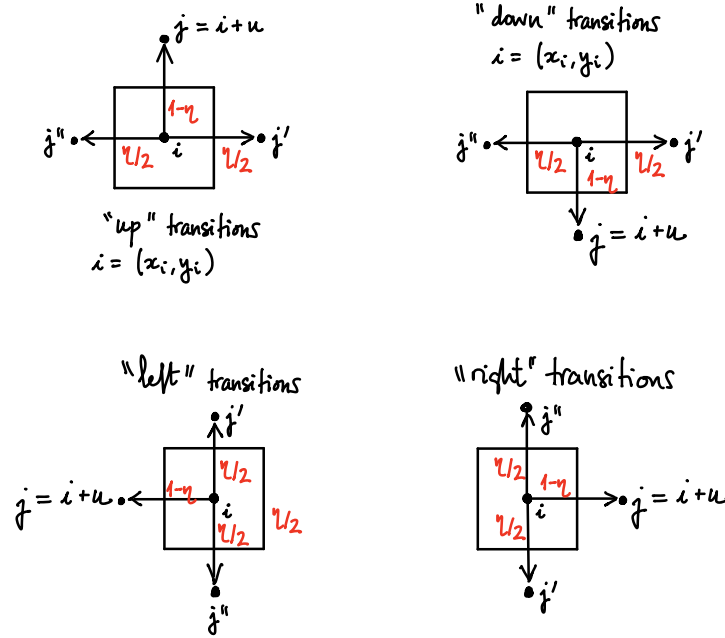


Figure 1: Graphical illustration of probabilities p when there are no obstacles. When $\eta = 1$, the transitions from the policy are deterministic. For $\eta < 1$, the transitions from the policy are random. Obstacles and readjusting of the policy are handled by the programs.

- (b) (10 points) Complete the function `valueIteration`. The iterations should use synchronous sweeps. Then use `valueIteration` to find an optimal policy and value function for `SmallGrid`. For how long you need to iterate achieve convergence error $\epsilon = 1e-3$ with $\gamma = 0.9$ and $\eta = 0.2$? Does the number of iterations change for different value function initializations? Is the policy the same as in part (a)?
- (c) (5 points) Using `MediumGrid` with the default cost function, play with the discount factor γ and the noise parameter η to obtain different results in this environment. Can you find parameters such that your optimal policy from value iteration goes through the cell next to the start cell, given by (2,2)?
- (d) (15 points) Consider the `MediumGridBridge` layout. This grid has the same layout as `MediumGrid`, but has two special (goal) states with positive payoff at (5,3) and at (3,3). The bottom row of the environment has states with negative payoff of -10 (cliff states). The starting state is at (1,2). We call the state (2,2) the “bridge” between the start and the cliff states. We distinguish between two types of paths: (1) paths that “risk the cliff” and travel near the bottom row of the grid; these paths are shorter but risk earning a large negative payoff. (2) Paths that “avoid the cliff” and travel along the top edge of the grid. These paths are longer but are less likely to incur huge negative payoffs (or large costs.)

Give an assignment of parameter values for the discount γ , and noise value η in the actions which produce an optimal policy that results in the following paths:

- 1) Prefer the close exit (with cost -1), risking the cliff (with cost +10)
- 2) Prefer the close exit (with cost -1), but avoiding the cliff (with cost +10)
- 3) Prefer the distant exit (with cost -10), risking the cliff (with cost +10)
- 4) Prefer the distant exit (with cost -10), but avoiding the cliff (with cost +10)

Programming Instructions:

- You will fill in portions of `main.py` for solving this assignment. You can add more functions within this file. You should submit `main.py` with your code and comments. Please do not change the other files in this distribution or submit any of our original files other than `main.py`.
- The appropriate function calls used for evaluating your functions are:

1. `V, pi, iteration = valueIteration(gamma,eta,grid),`
2. `V, pi, i_eval, i_imprv = policyIteration(gamma,eta,grid),`
3. Fill in the `params` variable with your values of γ and η from parts ?? and ??,

The outputs `V` and `pi` are $n \times m$ sized arrays. The `pi` variable should consist of numbers $\{0, 1, 2, 3\}$ corresponding to the moves: right $(1, 0)$, up $(0, 1)$, left $(-1, 0)$ and down $(0, -1)$, in that order. The inputs include discount factor γ , probability η , and a grid world (an instance of either `SmallGrid`, `MediumGrid`, or `MediumGridBridge`). The `params` variable holds your answer for discount factor and probability for each of the questions. Please modify the values as you see fit. The outputs `values` and `policy` are $n \times m$ sized arrays. The `policy` variable should consist of numbers $\{0, 1, 2, 3\}$ corresponding to the moves: right $\{(1, 0)\}$, top $\{(0, 1)\}$, left $\{(-1, 0)\}$ and down $\{(0, -1)\}$ respectively.

- Your code will be verified for basic technical correctness.
- Getting Help:
 1. Some sample code has been provided towards the end of the `main.py` file to understand the use of functions. If you are stuck on something, please use office hours and Piazza. Again, please start with the programming assignment early.